CURSO DE ESPECIALIZAÇÃO EM DEEP LEARNING · MÓDULO 3

Tarefa 3 — Implementação e Comparação de Arquiteturas para Visão Computacional

Relatório de desenvolvimento, resultados e análise comparativa: CNN, ResNet50 e redes recorrentes (LSTM e GRU)

Pedro Serrano · setembro de 2026

RESULTADOS PRINCIPAIS · FASHION-MNIST (10 000 IMAGENS DE TESTE)

PARTE 1 · CNN SIMPLES

92,6 %

acurácia no teste (IC 95 %: ± 0,5 p.p.)

421 642 parâmetros · **1,6 MB** de pesos
Treino: **4,8 min** em 19 épocas (CPU)
Inferência: 0,099 ms por imagem

PARTE 2 · RESNET50 PRÉ-TREINADA

92,1 %

acurácia no teste (IC 95 %: ± 0,5 p.p.)

23 608 202 parâmetros · **90,1 MB** de pesos
Treino: **75,6 min** em 7 épocas (CPU)
Inferência: 1,613 ms por imagem

PARTE 3 · LSTM

88,4 %

acurácia no teste (IC 95 %: ± 0,6 p.p.)

81 674 parâmetros · **0,3 MB** de pesos
Treino: **3,8 min** em 17 épocas (CPU)
Inferência: 0,156 ms por imagem

PARTE 3 · GRU

88,6 %

acurácia no teste (IC 95 %: ± 0,6 p.p.)

61 962 parâmetros · **0,2 MB** de pesos
Treino: **4,1 min** em 15 épocas (CPU)
Inferência: 0,093 ms por imagem

Este relatório acompanha o notebook *Tarefa3_Arquiteturas_Visao_Computacional_PedroSerrano.ipynb*, que contém o código comentado e todos os gráficos. As figuras, tabelas e capturas aqui apresentadas foram geradas a partir da execução completa desse notebook (90 minutos em CPU).

Python 3.12.6 · TensorFlow 2.21.0 · Keras 3.15.1 · NumPy 2.4.4 · pandas 3.0.2 · scikit-learn 1.8.0 · Matplotlib 3.10.8

Índice

1. Introdução 3

2. Ambiente e metodologia 4

3. Dados: Fashion-MNIST 5

4. Parte 1 — Rede convolucional (CNN) 6

4.1 Conceitos e arquitetura 6
4.2 Treino 7
4.3 Avaliação no conjunto de teste 7
4.4 O que a CNN “vê” 8

5. Parte 2 — Rede residual (ResNet50 com transfer learning) 9

5.1 Ligações residuais e ResNet50 9
5.2 Adaptações ao Fashion-MNIST 9
5.3 Treino em duas fases 11
5.4 Avaliação no conjunto de teste 12
5.5 ResNet50 face à CNN 13

6. Parte 3 — Redes recorrentes (LSTM e GRU) 13

6.1 A imagem como sequência 13
6.2 LSTM e GRU 14
6.3 Treino 15
6.4 Avaliação no conjunto de teste 15
6.5 Quanto da imagem precisa a RNN de ler? 16

7. Parte 4 — Comparação e análise 17

7.1 Tabela comparativa 17
7.2 Acurácia, dimensão e custo 18
7.3 Desempenho por classe 18
7.4 Imagens que nenhuma rede acerta 19
7.5 Em que contextos cada arquitetura é mais vantajosa 20

8. Conclusões 21

Anexo A — Capturas complementares do notebook 23

1. Introdução

Esta tarefa tem como objetivo implementar, treinar e comparar três famílias de redes neuronais usadas em visão computacional: uma **rede convolucional (CNN)** simples; uma **rede residual (ResNet50)** pré-treinada no ImageNet e adaptada por *transfer learning*; e uma **rede recorrente**, em duas variantes (**LSTM** e **GRU**), que lê cada imagem linha a linha, como uma sequência.

Para que a comparação seja justa, as quatro redes resolvem o mesmo problema — classificar as 10 classes de vestuário e calçado do Fashion-MNIST — com as mesmas partições de dados, as mesmas funções de treino e de avaliação e na mesma máquina. Assim, as diferenças de acurácia, de tempo de treino e de tamanho refletem as arquiteturas, e não o protocolo.

A Tabela 1 resume os requisitos do enunciado e a secção onde cada um é tratado. A secção 2 descreve o ambiente e a metodologia comum e a secção 3 os dados; as secções 4, 5 e 6 apresentam cada arquitetura (conceitos, implementação, treino e avaliação); a secção 7 compara-as e discute em que contextos cada uma é mais vantajosa; a secção 8 resume as conclusões. O anexo reúne capturas complementares do notebook.

***Tabela 1** — Requisitos do enunciado e onde são cumpridos*

Requisito do enunciado	Como foi cumprido	Secção
Parte 1 — dataset de imagens (CIFAR-10, MNIST ou Fashion-MNIST)	Fashion-MNIST: 70 000 imagens 28×28 em tons de cinzento, 10 classes	3
Pelo menos duas camadas convolucionais seguidas de *max-pooling*	`Conv2D(32, 3×3)` + `MaxPooling2D(2×2)` e `Conv2D(64, 3×3)` + `MaxPooling2D(2×2)`, com ReLU	4.1
Camada densa final com ativação Softmax	`Dense(10, activation="softmax")`, depois de `Dense(128, ReLU)` e *dropout*	4.1
Treinar e apresentar gráficos da evolução da perda e da acurácia	Figura 2; avaliação no teste (Tabela 6 e Figura 3)	4.2–4.3
Parte 2 — ResNet pré-treinada (p. ex. ResNet50 do `keras.applications`)	ResNet50 de `keras.applications`, com os pesos do ImageNet e uma cabeça nova para 10 classes	5.1
Aplicá-la ao mesmo dataset, podendo usar *transfer learning*	Mesmas partições; imagens adaptadas dentro do modelo; fase A com a base congelada e fase B de afinação	5.2–5.4
Comparar o desempenho com a CNN simples	Tabela 9 e discussão	5.5
Parte 3 — converter as imagens para um formato sequencial	Cada imagem é uma sequência de 28 passos (linhas), cada um com 28 píxeis	6.1
Implementar uma LSTM ou GRU	Implementadas as duas, com a mesma estrutura e o mesmo treino	6.2–6.3
Avaliar e comparar com a CNN e a ResNet	Tabela 11, Figura 11 e Tabela 12	6.4–6.5
Parte 4 — tabela comparativa (acurácia, tempo de treino, tamanho do modelo)	Tabela 12, com IC 95 %, parâmetros e latência; Figuras 12–15	7.1–7.4
Explicar em que contextos cada arquitetura é mais vantajosa	Tabela 14 e discussão	7.5
Entrega — .ipynb com código comentado, gráficos e tabelas	Notebook comentado; as figuras, a tabela (CSV) e os resultados (JSON) também são guardados em ficheiro	—
Documento PDF com capturas de ecrã, comparações e conclusões	Este relatório: 9 capturas do notebook, secção 7 e secção 8	7, 8, Anexo A

2. Ambiente e metodologia

O trabalho foi desenvolvido num notebook Jupyter, com Python 3.12.6, TensorFlow 2.21.0 (Keras 3.15.1) para as redes neuronais, scikit-learn 1.8.0 para a partição estratificada e as métricas por classe, pandas 3.0.2 e NumPy 2.4.4 para organizar os resultados e Matplotlib 3.10.8 para os gráficos. As versões exatas estão no ficheiro *requirements.txt* que acompanha o notebook.

O treino correu em **CPU** (8 núcleos lógicos, Windows 11): desde a versão 2.11, o TensorFlow nativo em Windows já não usa a GPU. É uma limitação que pesa sobretudo na ResNet50 e que condicionou duas escolhas — a resolução de entrada de 32×32 e o número de épocas da afinação (secção 5). A execução completa do notebook demorou 90 minutos, dos quais 76 no treino da ResNet50.

Para os resultados serem reprodutíveis, fixei a semente aleatória (42) com `keras.utils.set_random_seed`, que controla a partição dos dados, a inicialização dos pesos e o *dropout*. Mesmo assim, o paralelismo da CPU introduz pequenas diferenças numéricas entre execuções. Os quatro modelos seguiram o mesmo procedimento:

1. Partição dos dados em treino, validação e teste (Tabela 2). O conjunto de teste fica de parte até à avaliação final e é usado uma única vez por modelo.
2. Construção da rede com a API do Keras e compilação com o otimizador Adam e a entropia cruzada categórica (`sparse_categorical_crossentropy`, porque as etiquetas são inteiros de 0 a 9).
3. Treino com uma função comum, `treinar()`: mini-lotes, validação no fim de cada época e paragem antecipada (`EarlyStopping`) sobre a perda de validação, com reposição dos pesos da melhor época.
4. Avaliação com outra função comum, `avaliar()`: acurácia e perda no teste, intervalo de confiança, tempo de inferência, número de parâmetros e tamanho dos pesos (Tabela 4).

***Tabela 2** — Partição dos dados (comum aos quatro modelos)*

Conjunto	Imagens	Origem	Utilização
Treino	54 000	Treino oficial, menos a validação	Ajuste dos pesos
Validação	6 000	10 % do treino oficial, estratificada	Paragem antecipada e acompanhamento do sobreajuste
Teste	10 000	Teste oficial	Avaliação final, uma única vez por modelo

A Tabela 3 resume a configuração do treino. A CNN e as redes recorrentes usam exatamente os mesmos hiperparâmetros; a ResNet50 usa lotes mais pequenos (64), por ser muito mais pesada, e é treinada em duas fases, explicadas na secção 5.3.

***Tabela 3** — Configuração do treino de cada modelo (otimizador Adam; perda `sparse_categorical_crossentropy`)*

Modelo	Taxa de aprendizagem	Lote	Épocas (máx.)	Paciência	Parâmetros treinados
CNN	0,001	128	20	3	421 642
ResNet50 — fase A	0,001	64	2	3	20 490
ResNet50 — fase B	0,0001	64	5	2	23 501 962
LSTM	0,001	128	20	3	81 674
GRU	0,001	128	20	3	61 962

Todas as métricas são calculadas pela mesma função e no mesmo conjunto de teste (Tabela 4). O intervalo de confiança é importante para ler a comparação: com 10 000 imagens de teste, diferenças de acurácia abaixo

de 0,7–0,9 p.p. entre dois destes modelos (consoante o par) não permitem concluir que um é melhor do que o outro.

Tabela 4 *— Métricas usadas na avaliação e na comparação*

Métrica	Como é calculada
Acurácia (teste)	Proporção das 10 000 imagens de teste bem classificadas.
IC 95 %	±1,96·√(p(1 − p)/n), com n = 10 000: entre ±0,51 e ±0,63 p.p. nestes quatro modelos. Para a diferença entre dois modelos, a margem é $\sqrt{(IC_1^2 + IC_2^2)}$: entre ±0,74 e ±0,88 p.p.
Perda (teste)	Entropia cruzada média, calculada a partir das probabilidades previstas para a classe certa.
Precisão, recall e F1	Por classe: das imagens atribuídas à classe, quantas lhe pertencem (precisão); das imagens da classe, quantas foram encontradas (recall); média harmónica das duas (F1).
Tempo de treino	Tempo de relógio total do `fit`, incluindo a validação no fim de cada época; na ResNet50, soma das duas fases.
Parâmetros e tamanho	Número de pesos e memória que ocupam em `float32` (4 bytes por parâmetro), ou seja, o tamanho do modelo guardado sem o estado do otimizador.
Inferência	Tempo médio por imagem ao classificar o conjunto de teste em lotes de 256, depois de um lote de aquecimento.

3. Dados: Fashion-MNIST

O Fashion-MNIST tem 70 000 imagens de 28×28 píxeis em tons de cinzento, de 10 classes de vestuário e calçado (Figura 1), com 7 000 imagens por classe: 6 000 no treino e 1 000 no teste. O dataset é, portanto, equilibrado, e a acurácia é uma métrica adequada.

Dos três datasets sugeridos no enunciado, escolhi o Fashion-MNIST por ser o que melhor separa as arquiteturas sem tornar o treino impraticável em CPU. No MNIST (dígitos manuscritos), quase qualquer rede chega aos 98–99 %, e as diferenças entre modelos ficariam escondidas no ruído. O CIFAR-10 (fotografias a cores) exigiria muito mais épocas e aumento de dados, o que, em CPU e com quatro modelos, seria incomportável. O Fashion-MNIST é difícil o suficiente — há classes muito parecidas entre si, como camisa, T-shirt, camisola e casaco — e é o dataset da Tarefa 2, o que permite comparar com a MLP aí treinada (89,16 % de acurácia no teste).



Figura 1 *— Três exemplos de cada classe do Fashion-MNIST (conjunto de treino).*

Os píxeis foram normalizados de 0–255 para 0–1 (`float32`). A CNN e a ResNet50 recebem as imagens com um canal, com a forma `(28, 28, 1)`; as redes recorrentes recebem-nas sem o canal, `(28, 28)`, como sequências de 28 linhas. A validação foi estratificada: as 6 000 imagens mantêm exatamente 10 % de cada classe.

4. Parte 1 — Rede convolucional (CNN)

4.1 Conceitos e arquitetura

Uma CNN aproveita duas propriedades das imagens: os padrões relevantes são **locais** (uma aresta, um contorno, uma textura) e podem aparecer **em qualquer posição**. Por isso, em vez de ligar cada píxel a cada neurónio, como a MLP da Tarefa 2, combina três tipos de camadas:

- **Convolução** — cada filtro 3×3 percorre a imagem e responde a um padrão local. Os mesmos pesos são usados em todas as posições (*partilha de pesos*), o que reduz muito o número de parâmetros e faz com que um padrão seja reconhecido onde quer que apareça.
- ***Max-pooling*** — reduz a resolução para metade, guardando o valor máximo de cada janela 2×2. Dá tolerância a pequenas deslocações e alarga o campo recetivo das camadas seguintes, que passam a combinar padrões de zonas maiores da imagem.
- **Camadas densas** — combinam as características extraídas; a Softmax final converte-as em probabilidades para as 10 classes.

A rede tem dois blocos convolucionais, cada um seguido de *max-pooling*, como pede o enunciado (Tabela 5). O primeiro bloco, com 32 filtros, deteta padrões simples; o segundo, com 64 filtros sobre mapas de 14×14, combina-os em formas mais complexas. O preenchimento `padding="same"` mantém o tamanho dos mapas em cada convolução, e são os *poolings* que o reduzem (28 → 14 → 7). Os dois *dropouts* de 30 % desligam aleatoriamente valores durante o treino, para reduzir o sobreajuste.

Tabela 5 *— Arquitetura da CNN*

Camada	Tipo	Configuração	Saída	Parâmetros
`conv1`	Conv2D	32 filtros 3×3, ReLU	28 × 28 × 32	320
`pool1`	MaxPooling2D	janela 2×2	14 × 14 × 32	0
`conv2`	Conv2D	64 filtros 3×3, ReLU	14 × 14 × 64	18 496
`pool2`	MaxPooling2D	janela 2×2	7 × 7 × 64	0
`achatar`	Flatten	7 × 7 × 64 = 3 136 valores	3 136	0
`dropout1`	Dropout	30 %	3 136	0
`densa`	Dense	128 neurónios, ReLU	128	401 536
`dropout2`	Dropout	30 %	128	0
`saida`	Dense	10 neurónios, Softmax	10	1 290
Total				**421 642**

As duas camadas convolucionais têm apenas 18 816 parâmetros (4,5 % do total), precisamente por causa da partilha de pesos; 95 % estão na camada densa de 128 neurónios, que recebe os 3 136 valores do último mapa de características. No total são 421 642 parâmetros (1,6 MB em `float32`) — mais do que os 235 146 da MLP da Tarefa 2; a diferença de desempenho, como se verá, está na forma como os parâmetros são usados.

In [7]:
```
# ============================================================================
# 2.1  Arquitetura da CNN
# ============================================================================
def criar_cnn():
    return keras.Sequential([
        keras.Input(shape=(28, 28, 1), name="imagem"),
        # Bloco 1: 32 filtros 3x3 detetam padrões simples (arestas, contornos)
        layers.Conv2D(32, 3, padding="same", activation="relu", name="conv1"),
        layers.MaxPooling2D(2, name="pool1"),                    # 28x28 -> 14x14
        # Bloco 2: 64 filtros combinam os padrões anteriores em formas mais complexas
        layers.Conv2D(64, 3, padding="same", activation="relu", name="conv2"),
        layers.MaxPooling2D(2, name="pool2"),                    # 14x14 -> 7x7
        # Classificador
        layers.Flatten(name="achatar"),                           # 7x7x64 -> 3 136 valores
        layers.Dropout(0.3, name="dropout1"),
        layers.Dense(128, activation="relu", name="densa"),
        layers.Dropout(0.3, name="dropout2"),
        layers.Dense(10, activation="softmax", name="saida"),    # probabilidades das 10 classes
    ], name="CNN")


cnn = criar_cnn()
cnn.compile(optimizer=keras.optimizers.Adam(1e-3), loss="sparse_categorical_crossentropy",
            metrics=["accuracy"])
cnn.summary()
```

Captura 1 *— definição e compilação da CNN no notebook (`criar_cnn`).*

4.2 Treino

O treino correu 19 épocas, em 4,8 min (15 s por época). A Figura 2 mostra as curvas de treino e de validação: a perda de validação desceu de 0,334 na 1.ª época para 0,187 na 16.ª; como não voltou a melhorar nas 3 épocas seguintes, a paragem antecipada interrompeu o treino e repôs os pesos da 16.ª época.

Na melhor época, a acurácia de treino era de 94,60 % e a de validação de 93,13 %: uma folga de 1,47 p.p., pequena. O *dropout* e a paragem antecipada mantiveram o sobreajuste sob controlo — a perda de treino continua a descer depois da melhor época, mas a de validação estabiliza, que é o sinal clássico de que continuar a treinar já só serviria para memorizar o treino.

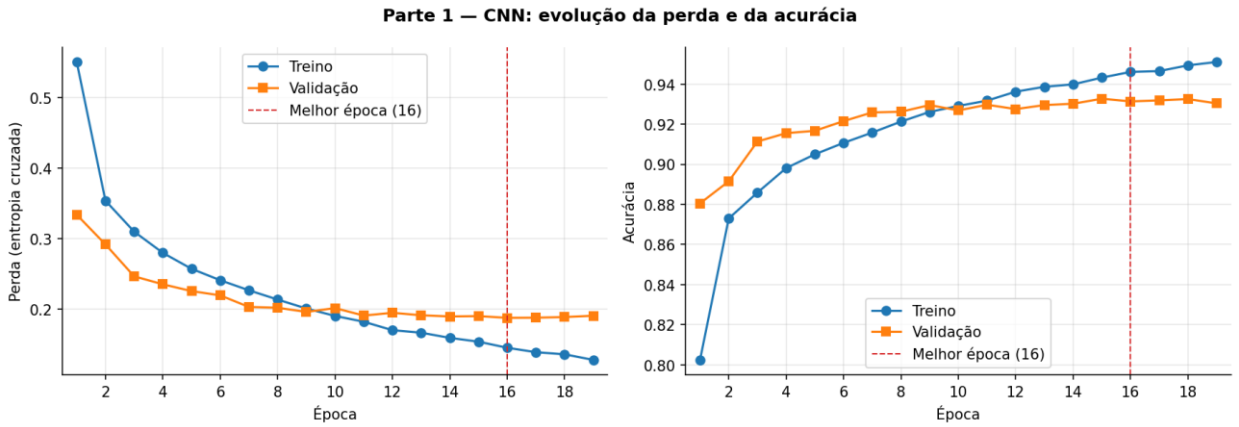


Figura 2 *— Evolução da perda e da acurácia da CNN, no treino e na validação; a linha tracejada marca a melhor época.*

Nota: as métricas de treino são medidas com o *dropout* ativo e em média ao longo de cada época, enquanto a rede ainda está a aprender; por isso, nas primeiras épocas, a acurácia de validação chega a ficar acima da de treino.

4.3 Avaliação no conjunto de teste

No conjunto de teste, a CNN acertou **92,62 %** das imagens (IC 95 %: ± 0,51 p.p.), com uma perda de 0,2180. É uma melhoria de 3,46 p.p. face à MLP da Tarefa 2 (89,16 %), treinada com a mesma partição dos dados: explorar a estrutura espacial da imagem compensa. A Tabela 6 detalha a precisão, o recall e o F1 de cada classe.

Tabela 6 *— Métricas por classe da CNN no conjunto de teste (a sombreado, as três classes com pior F1)*

Classe	Precisão	Recall	F1	Imagens
T-shirt/top	0,843	0,906	0,873	1 000
Calças	0,992	0,986	0,989	1 000
Camisola	0,892	0,888	0,890	1 000
Vestido	0,907	0,958	0,932	1 000
Casaco	0,877	0,894	0,886	1 000
Sandália	0,991	0,985	0,988	1 000
Camisa	0,827	0,714	0,767	1 000
Sapatilha	0,965	0,975	0,970	1 000
Mala	0,991	0,983	0,987	1 000
Botim	0,971	0,973	0,972	1 000
Média (macro)	**0,926**	**0,926**	**0,925**	**10 000**

A matriz de confusão (Figura 3) mostra onde a rede falha. **Camisa** é a classe mais difícil (F1 = 0,767; recall de 71,4 %): as camisas são confundidas sobretudo com T-shirts (13 % das camisas do teste) e, em menor grau, com casacos (7 %) e camisolas (5 %) — peças com a mesma silhueta que, a 28×28 píxeis, se distinguem apenas por pormenores como a gola, os botões ou o comprimento das mangas. Seguem-se T-shirt/top e casaco, precisamente as classes com que a camisa se confunde. No extremo oposto, calças, sandália, mala e botim ficam entre 97 e 99 % de F1, porque têm formas únicas.

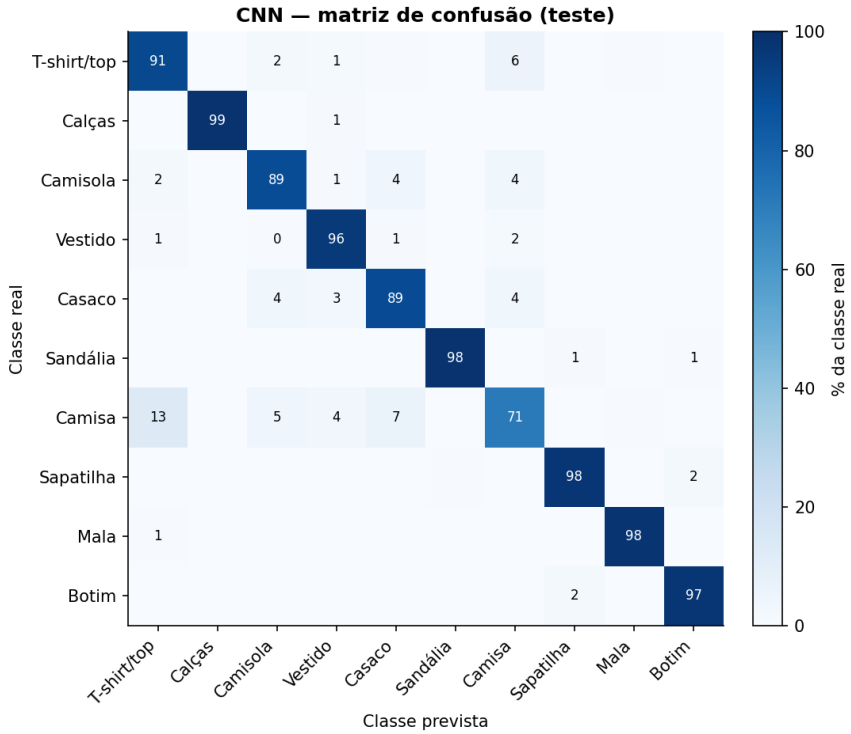


Figura 3 *— Matriz de confusão da CNN no teste: percentagem das imagens de cada classe real atribuída a cada classe prevista.*

4.4 O que a CNN “vê”

A Figura 4 mostra a resposta dos 8 filtros mais ativos de cada camada convolucional a um botim do conjunto de teste. Na primeira camada (28×28), cada filtro responde a um padrão simples: uns realçam o contorno superior da peça (filtros 25, 26 e 29), outros a sola (13 e 14), o cano e o calcanhar (16 e 20) ou a área inteira

do botim (30). Na segunda (14×14), as respostas são mais esparsas e localizadas — a sola, a linha do peito do pé, o calcanhar —, porque cada filtro combina vários padrões da camada anterior. É esta hierarquia, de padrões simples para padrões compostos, que redes mais profundas, como a ResNet50, levam muito mais longe.

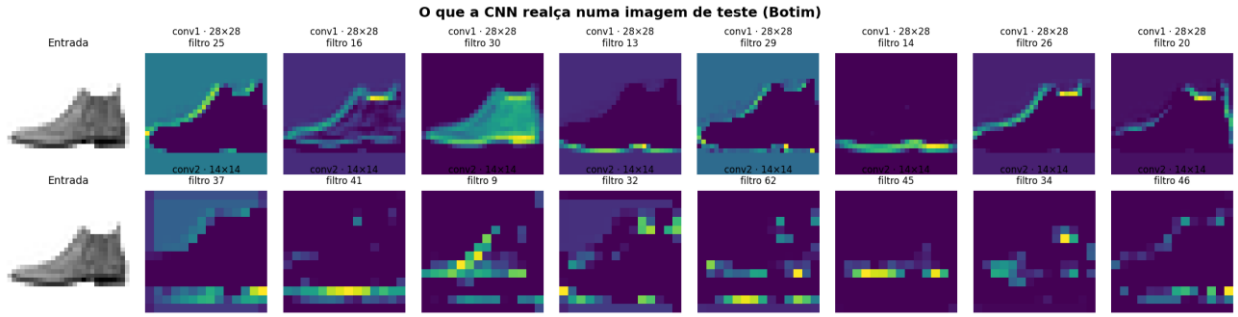


Figura 4 *— Mapas de ativação dos 8 filtros mais ativos de cada camada convolucional para um botim do conjunto de teste.*

5. Parte 2 — Rede residual (ResNet50 com transfer learning)

5.1 Ligações residuais e ResNet50

Acrescentar camadas a uma rede devia permitir-lhe aprender funções mais ricas, mas, a partir de certa profundidade, as redes convencionais treinam *pior*: o próprio erro de treino sobe. É o problema da **degradação** — o sinal e o gradiente diluem-se ao atravessar dezenas de camadas. A ResNet resolve-o com **ligações residuais** (atalhos): cada bloco aprende apenas uma correção F(x) e soma-a à própria entrada, y = F(x) + x. Se a correção não for útil, basta aprender F(x) ≈ 0, e o gradiente tem sempre um caminho direto até às primeiras camadas.

A **ResNet50** tem 50 camadas com pesos: uma convolução inicial 7×7, 16 blocos *bottleneck* (1×1 → 3×3 → 1×1) distribuídos por quatro estágios de 3, 4, 6 e 3 blocos, e a camada de classificação. Na implementação do Keras, a base convolucional sem a classificação tem 176 camadas — 53 convolucionais, das quais 49 no caminho principal e 4 nas projeções dos atalhos — e 23 587 712 parâmetros (23,6 milhões, 56× os da CNN).

Usei os pesos treinados no **ImageNet** (1,2 milhões de fotografias de 1 000 classes), disponíveis em `keras.applications`, e substituí a camada de classificação original por uma cabeça nova para as 10 classes do Fashion-MNIST. É isto o *transfer learning*: reaproveitar os filtros aprendidos num problema grande e ajustá-los a um problema novo, em vez de treinar a rede de raiz.

5.2 Adaptações ao Fashion-MNIST

A ResNet50 foi treinada com fotografias RGB de 224×224 píxeis, pré-processadas de uma forma específica, por isso as imagens do Fashion-MNIST têm de ser adaptadas (Tabela 7). Fiz as adaptações **dentro do modelo**, com camadas do Keras (Captura 2), por duas razões: as imagens ampliadas nunca chegam a ocupar memória, porque a ampliação é feita lote a lote; e o modelo final recebe diretamente as imagens originais, sem pré-processamento externo que alguém se possa esquecer de aplicar.

***Tabela 7** — Adaptações das imagens à ResNet50 pré-treinada, feitas dentro do modelo*

Diferença	Solução	Camada Keras
Resolução — a ResNet50 do Keras exige pelo menos 32×32 píxeis	Ampliação de 28×28 para 32×32, lote a lote	`Resizing(32, 32)`
Canais — a rede espera imagens RGB	O canal cinzento é replicado três vezes	`Concatenate([x, x, x])`
Pré-processamento — o do treino original no ImageNet (modo "caffe")	Escala 0–255 e subtração da média de cada canal do ImageNet (103,9; 116,8; 123,7)	`Rescaling(255)` + `Normalization`
Classes — a cabeça original classifica 1 000 classes	Sem a cabeça original; *pooling* médio global (2 048 valores), *dropout* de 30 % e `Dense(10, Softmax)`	`include_top=False, pooling="avg"`

A resolução foi o compromisso mais difícil. Ampliar para 224×224 aproximaria as imagens das do ImageNet, mas cada época ficaria cerca de 49 vezes mais cara (224^2 / 32^2), o que em CPU é impraticável; e ampliar o dataset inteiro antes do treino ocuparia cerca de 32 GB de memória (54 000 × 224 × 224 × 3 × 4 bytes). Fiquei pelos 32×32, o mínimo que a ResNet50 do Keras aceita.

Para garantir que o pré-processamento dentro do modelo é exatamente o que a rede espera, comparei a saída dessas camadas com a função oficial `keras.applications.resnet50.preprocess_input` (Captura 3): a diferença máxima é zero — as duas versões são idênticas.

In [12]:
```python
# ============================================================================
# 3.1  ResNet50 pré-treinada no ImageNet + nova cabeça de classificação
# ============================================================================
TAMANHO_RESNET = 32   # mínimo aceite pela ResNet50 do Keras; com GPU pode experimentar-se 64 ou 96
MEDIAS_IMAGENET = [103.939, 116.779, 123.68]   # média de cada canal (ordem BGR) no treino original


def criar_resnet(tamanho=TAMANHO_RESNET):
    """Devolve (modelo completo, base ResNet50). A base começa congelada (fase A)."""
    entrada = keras.Input(shape=(28, 28, 1), name="imagem")
    # (1) 28x28 -> tamanho x tamanho, dentro do modelo: as imagens ampliadas nunca ocupam memória
    x = layers.Resizing(tamanho, tamanho, name="redimensionar")(entrada)
    # (2) 1 canal -> 3 canais, porque a rede foi treinada com imagens RGB
    x = layers.Concatenate(name="cinzento_para_rgb")([x, x, x])
    # (3) pré-processamento do ImageNet ("caffe"): escala 0-255 e subtração da média de cada canal
    x = layers.Rescaling(255.0, name="escala_0_255")(x)
    x = layers.Normalization(mean=MEDIAS_IMAGENET, variance=[1.0, 1.0, 1.0], name="subtrair_medias")(x)
    # (4) base convolucional pré-treinada, sem a classificação original (1 000 classes);
    #     pooling="avg" faz a média global de cada mapa de características -> vetor de 2 048 valores
    base = keras.applications.ResNet50(include_top=False, weights="imagenet",
                                       input_shape=(tamanho, tamanho, 3), pooling="avg")
    base.trainable = False
    x = base(x, training=False)   # training=False: BatchNormalization sempre em modo de inferência
    # (5) nova cabeça para as 10 classes do Fashion-MNIST
    x = layers.Dropout(0.3, name="dropout")(x)
    saida = layers.Dense(10, activation="softmax", name="saida")(x)
    return keras.Model(entrada, saida, name="ResNet50_transfer_learning"), base


resnet, base_resnet = criar_resnet()
resnet.compile(optimizer=keras.optimizers.Adam(1e-3), loss="sparse_categorical_crossentropy",
               metrics=["accuracy"])
resnet.summary()

n_conv = sum(isinstance(camada, layers.Conv2D) for camada in base_resnet.layers)
print(f"Base ResNet50: {len(base_resnet.layers)} camadas Keras, das quais {n_conv} convolucionais "
      f"(49 no caminho principal + 4 projeções nos atalhos); {base_resnet.count_params():,} parâmetros")
```

***Captura 2** — construção do modelo: adaptações, base ResNet50 congelada e nova cabeça (`criar_resnet`).*

In [13]:
```
# 3.2 Verificação: o pré-processamento dentro do modelo é igual ao preprocess_input oficial
from keras.applications.resnet50 import preprocess_input

pre_modelo = keras.Model(resnet.input, resnet.get_layer("subtrair_medias").output)
amostra = X_TREINO[:32]
nosso = pre_modelo.predict(amostra, verbose=0)
ampliada = keras.ops.convert_to_numpy(keras.ops.image.resize(amostra, (TAMANHO_RESNET, TAMANHO_RESNET)))
oficial = preprocess_input(np.repeat(ampliada, 3, axis=-1) * 255.0)
diferenca = float(np.abs(nosso - oficial).max())
print(f"Diferença máxima entre as duas versões: {diferenca:.1e} →",
      "equivalentes ✔" if diferenca < 1e-3 else "DIFERENTES ✘ (rever o pré-processamento)")
```

```
Diferença máxima entre as duas versões: 0.0e+00 → equivalentes ✔
```

Captura 3 *— verificação de que o pré-processamento dentro do modelo é igual ao* `preprocess_input` *oficial.*

5.3 Treino em duas fases

Se a base e a cabeça fossem treinadas em conjunto desde o início, os gradientes grandes de uma cabeça ainda aleatória propagar-se-iam pela base e estragariam os pesos pré-treinados. Por isso, o treino foi feito em duas fases:

- **Fase A — extração de características.** A base fica congelada (`base.trainable = False`) e só a cabeça nova é treinada. A ResNet50 funciona como um extrator fixo de características do ImageNet.
- **Fase B — afinação (*fine-tuning*).** A base é descongelada e o treino continua com uma taxa de aprendizagem 10 vezes menor (0,0001), para ajustar os filtros às peças de roupa sem os destruir. As camadas *BatchNormalization* ficam congeladas e em modo de inferência (`training=False`), para manterem as estatísticas do ImageNet em vez de as trocarem pelas de lotes pequenos.

Tabela 8 *— As duas fases do treino da ResNet50*

Fase	Parâmetros treinados	Taxa de aprendizagem	Épocas	Tempo	Por época	Acurácia de validação (melhor época)
A — base congelada	20 490	0,001	2	3,9 min	1,9 min	83,77 %
B — afinação	23 501 962	0,0001	5	71,8 min	14,4 min	92,98 %

A Tabela 8 e a Figura 5 resumem as duas fases. Com a base congelada, a cabeça nova chegou a apenas 83,77 % de acurácia de validação, muito abaixo dos 93,13 % da CNN: as características do ImageNet, calculadas sobre silhuetas cinzentas de 32×32, separam as classes pior do que os filtros que a CNN aprendeu de raiz. A afinação muda o quadro — logo na primeira época da fase B, a acurácia de validação sobe para 90,28 %, e o melhor resultado foi de 92,98 % na 6.ª época (a 4.ª da fase B).

Na época seguinte, a perda de validação voltou a subir (0,219, contra 0,200) enquanto a de treino continuava a descer e a acurácia de treino passava os 94 % — o primeiro sinal de sobreajuste. Como a paragem antecipada repõe os pesos da melhor época no fim do treino, o modelo avaliado é o da 6.ª época.

O preço está no tempo: cada época da fase B demorou entre 12,7 min e 15,8 min (71,8 min no total), contra 1,9 min na fase A, porque passa a ser preciso calcular gradientes para 23,5 milhões de parâmetros. Foi por isso que limitei a fase B a 5 épocas; como a validação já piorou na última, mais épocas sozinhas não teriam ajudado.

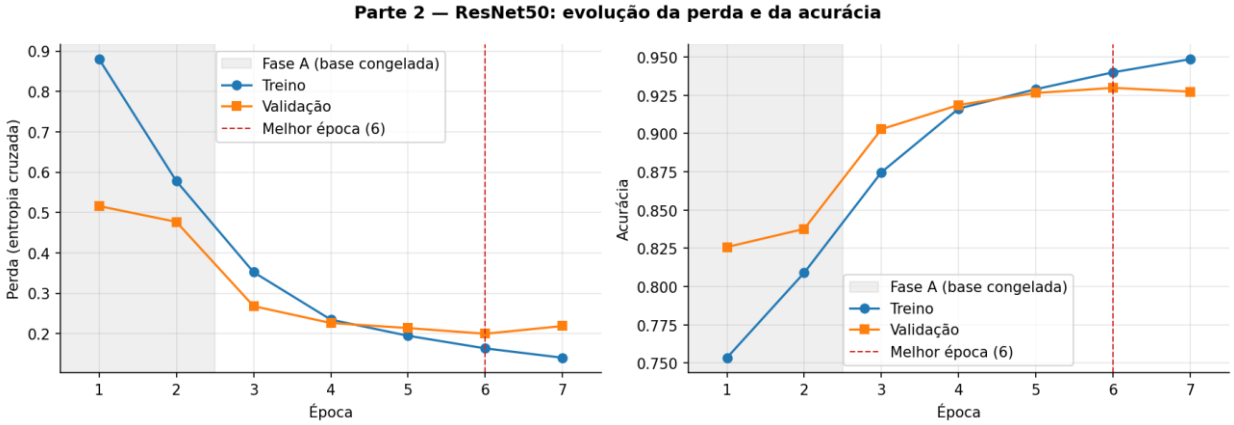


Figura 5 — Evolução da perda e da acurácia da ResNet50; o fundo cinzento marca a fase A (base congelada).

5.4 Avaliação no conjunto de teste

No conjunto de teste, a ResNet50 acertou **92,14 %** das imagens (IC 95 %: ± 0,53 p.p.), com uma perda de 0,2241 e um F1 médio de 0,921. A matriz de confusão (Figura 6) tem o mesmo padrão da CNN: a classe mais difícil é a camisa (F1 = 0,770, contra 0,767 na CNN), seguida de T-shirt/top, casaco e camisola. As duas redes tropeçam nas mesmas classes, o que indica que o limite está sobretudo nos dados — silhuetas de 28×28 muito parecidas — e não apenas na arquitetura.

Os erros não são, porém, exatamente os mesmos. A ResNet50 atribui mais imagens à classe casaco (recall de 93,5 %, contra 89,4 % na CNN), mas à custa da precisão (81,9 % contra 87,7 %), sobretudo porque lhe junta camisolas: o recall da camisola cai para 81,9 % (88,8 % na CNN).

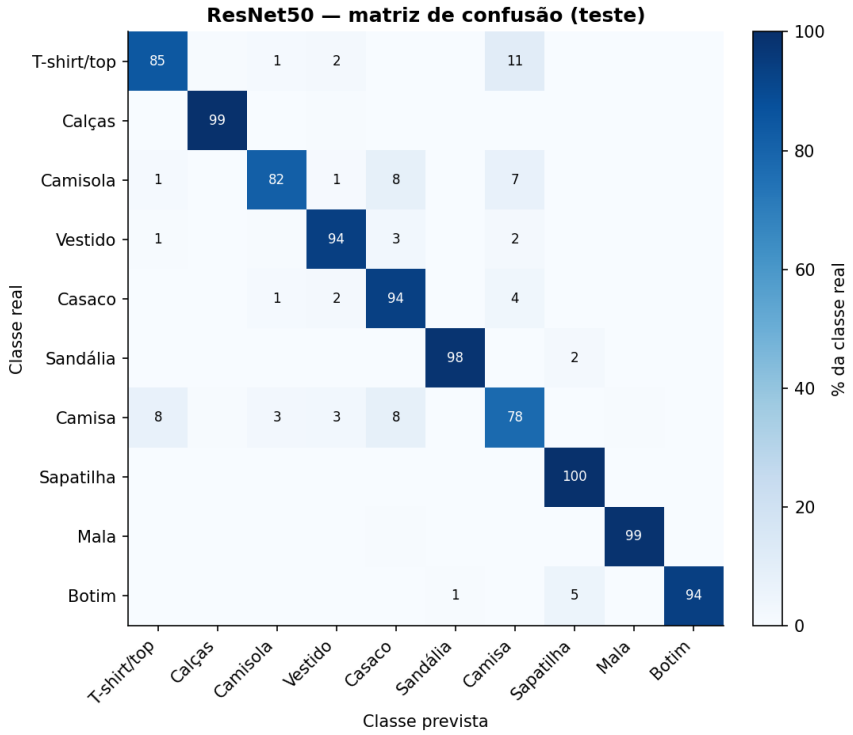


Figura 6 — Matriz de confusão da ResNet50 no teste.

5.5 ResNet50 face à CNN

Tabela 9 — ResNet50 face à CNN simples

	CNN	ResNet50	ResNet50 / CNN
Acurácia no teste	92,62 %	92,14 %	−0,48 p.p.
Parâmetros	421 642	23 608 202	56×
Tamanho dos pesos	1,6 MB	90,1 MB	56×
Tempo de treino	4,8 min	75,6 min	15,7×
Inferência (ms/imagem)	0,099	1,613	16,3×

A Tabela 9 responde à pergunta do enunciado: a diferença de acurácia (−0,48 p.p.) está dentro da margem de erro (≈ ± 0,7 p.p.): na prática, as duas redes empatam, mas com 56× os parâmetros, 56× o tamanho, 15,7× o tempo de treino e 16,3× o tempo de inferência. Uma rede 56 vezes maior não ganha automaticamente a uma CNN pequena; neste problema, há três fatores que jogam contra a ResNet50:

1. **Resolução.** A 32×32, a ResNet50 reduz a imagem 32 vezes e o último estágio trabalha sobre mapas de 1×1: grande parte da profundidade da rede é desperdiçada, porque os filtros do ImageNet foram aprendidos para objetos vistos a 224×224.
2. **Domínio.** Silhuetas de roupa em tons de cinzento, centradas e sem fundo, estão muito longe das fotografias a cores do ImageNet, por isso as características pré-treinadas transferem-se só em parte — como mostra a fase A — e é a afinação da base inteira que as adapta.
3. **Quantidade de dados.** Com 54 000 imagens de treino, uma CNN pequena aprende de raiz o que precisa; o *transfer learning* dá mais vantagem quando há poucos dados rotulados.

O resultado da ResNet50 não é mau — 92,14 % fica 3,0 p.p. acima da MLP da Tarefa 2 e mostra que a afinação consegue adaptar os filtros do ImageNet a um domínio muito diferente. O que lhe falta é uma vantagem que pague o custo: neste problema, a CNN chega ao mesmo sítio com 16× menos tempo de treino e 56× menos memória.

6. Parte 3 — Redes recorrentes (LSTM e GRU)

6.1 A imagem como sequência

Uma rede recorrente processa uma sequência elemento a elemento e mantém um **estado interno** que resume o que já viu. Para a aplicar a imagens, como pede o enunciado, cada imagem passa a ser uma sequência de **28 passos de tempo** — as linhas, de cima para baixo —, cada um com **28 características**, os píxeis dessa linha (Figura 7). A conversão é imediata: basta retirar o canal, passando de `(28, 28, 1)` para `(28, 28)`. A rede lê a imagem como quem lê um texto, uma linha de cada vez, sem nunca a ver inteira; no fim, só o último estado alimenta a classificação.

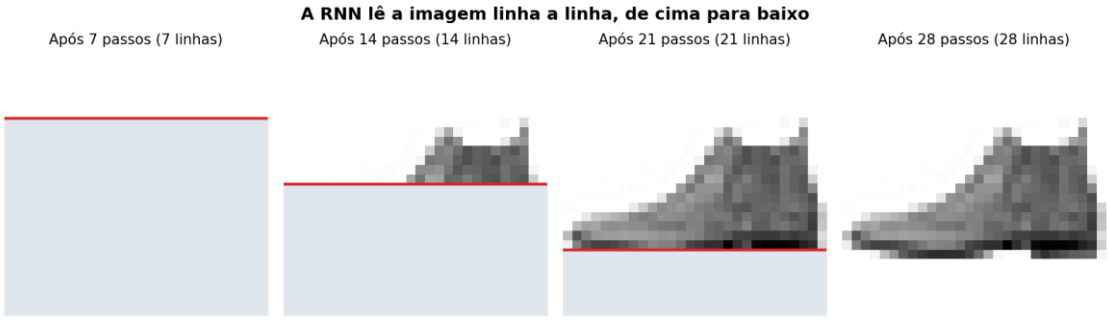


Figura 7 — O que a rede recorrente já leu de uma imagem de teste ao fim de 7, 14, 21 e 28 passos.

6.2 LSTM e GRU

Uma RNN simples sofre de **gradiente evanescente**: ao retropropagar o erro por muitos passos, o gradiente é multiplicado repetidamente pelos mesmos pesos e tende a desaparecer, pelo que a rede esquece o início da sequência. As duas variantes pedidas resolvem isto com **portas** — pequenas camadas sigmoides que decidem, em cada passo, que informação guardar e que informação esquecer:

- **LSTM** (*Long Short-Term Memory*) — tem uma **célula de memória** separada do estado oculto e três portas: esquecimento (o que apagar da memória), entrada (o que acrescentar) e saída (o que expor no estado). Como a memória é atualizada por somas, o gradiente consegue atravessar muitos passos.
- **GRU** (*Gated Recurrent Unit*) — versão simplificada, com duas portas (atualização e *reset*) e sem célula de memória separada. Tem menos parâmetros e costuma treinar mais depressa, com resultados semelhantes.

O enunciado pede uma LSTM *ou* uma GRU; implementei as duas, com a mesma estrutura — `RNN(128)` → `Dropout(0,3)` → `Dense(10, Softmax)` — e o mesmo treino da CNN, para isolar o efeito da célula recorrente (Tabela 10 e Captura 4). A camada recorrente aceita sequências de qualquer comprimento, o que permite a análise da secção 6.5.

Tabela 10 *— Parâmetros das duas redes recorrentes (entrada: 28 passos × 28 características)*

Camada	LSTM	GRU	Como se calcula
Recorrente, 128 unidades	80 384	60 672	LSTM: 4 × (128 × (28 + 128) + 128) GRU: 3 × (128 × (28 + 128) + 2 × 128)
`Dropout(0,3)`	0	0	—
`Dense(10, Softmax)`	1 290	1 290	128 × 10 + 10
Total	**81 674**	**61 962**	a GRU tem 24 % menos parâmetros

A GRU tem três conjuntos de pesos (duas portas e o estado candidato) contra os quatro da LSTM (três portas e a memória candidata), daí a diferença de parâmetros; o termo 2 × 128 vem de o Keras usar, por omissão, dois vetores de *bias* na GRU (`reset_after=True`), a variante compatível com a implementação cuDNN. Pelo mesmo motivo, deixei a camada recorrente com os parâmetros por omissão: em GPU, o Keras usa então o cuDNN, muito mais rápido, e opções como `recurrent_dropout` desligá-lo-iam.

In [19]:
```python
# 4.2 LSTM
def criar_rnn(tipo, unidades=128):
    """RNN(unidades) -> Dropout -> Dense(10, Softmax); só o último estado alimenta a classificação."""
    camada_recorrente = {"LSTM": layers.LSTM, "GRU": layers.GRU}[tipo]
    return keras.Sequential([
        # None: aceita sequências de qualquer comprimento (usado na análise da secção 4.4)
        keras.Input(shape=(None, 28), name="linhas"),
        # Parâmetros por omissão de propósito: em GPU, o Keras usa a implementação cuDNN,
        # muito mais rápida; opções como recurrent_dropout desligam-na
        camada_recorrente(unidades, name=tipo.lower()),
        layers.Dropout(0.3, name="dropout"),
        layers.Dense(10, activation="softmax", name="saida"),
    ], name=tipo)


lstm = criar_rnn("LSTM")
lstm.compile(optimizer=keras.optimizers.Adam(1e-3), loss="sparse_categorical_crossentropy",
             metrics=["accuracy"])
lstm.summary()
hist_lstm, tempo_lstm, tempos_lstm = treinar(lstm, X_TREINO_SEQ, y_treino, X_VAL_SEQ, y_val,
                                             epocas=EPOCAS_MAX, lote=128)
print(f"\nTreino da LSTM: {tempo_lstm:.0f} s em {len(tempos_lstm)} épocas "
      f"({np.mean(tempos_lstm):.1f} s por época)")
```

Captura 4 *— definição comum das duas redes recorrentes (`criar_rnn`) e treino da LSTM.*

6.3 Treino

A LSTM treinou 17 épocas (melhor: 14.ª) e a GRU 15 (melhor: 12.ª), em 3,8 min e 4,1 min, respetivamente (Figuras 8 e 9). As curvas têm a mesma forma das da CNN, mas começam mais abaixo e sobem mais devagar: na 1.ª época, a acurácia de validação era de 79,95 % na LSTM e 79,80 % na GRU, contra 88,05 % na CNN. Na melhor época, a folga entre treino e validação foi de 0,28 p.p. na LSTM e negativa na GRU (a validação ficou 0,35 p.p. acima do treino): com 128 unidades e *dropout*, nenhuma das duas chegou a sobreajustar de forma visível.

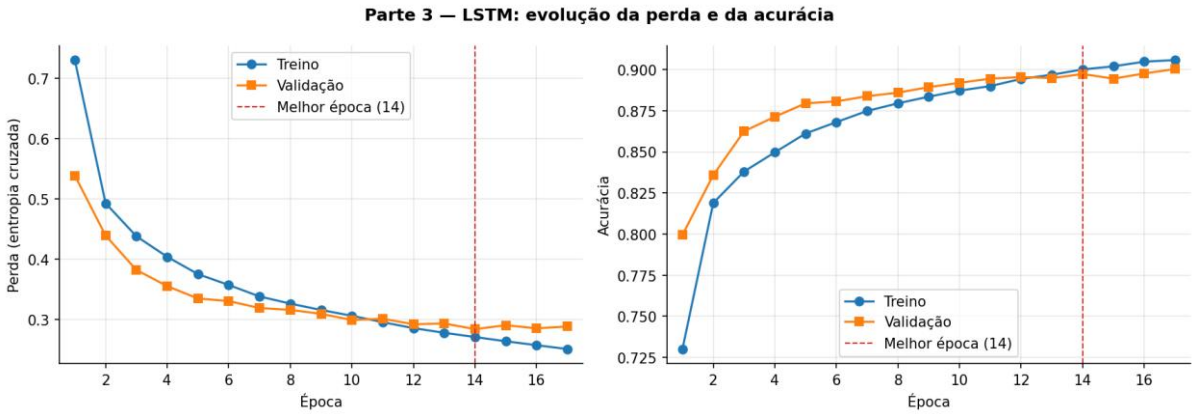


Figura 8 — Evolução da perda e da acurácia da LSTM.

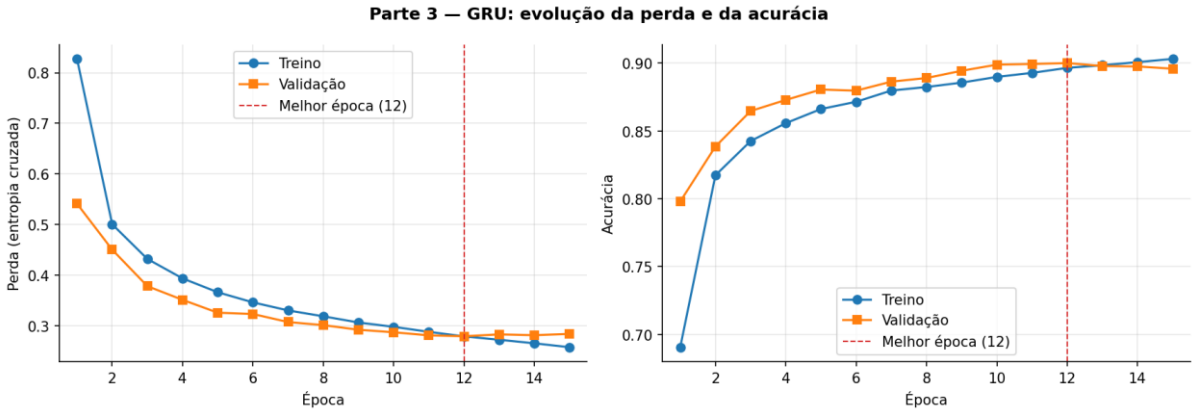


Figura 9 — Evolução da perda e da acurácia da GRU.

6.4 Avaliação no conjunto de teste

Tabela 11 — LSTM face à GRU: métricas no teste, custo de treino e de inferência

Métrica	LSTM	GRU	Diferença (LSTM – GRU)
Acurácia no teste	88,45 % (± 0,63)	88,61 % (± 0,62)	−0,16 p.p.
Perda no teste	0,3240	0,3151	0,0089
F1 médio (macro)	0,884	0,886	−0,002
Épocas (melhor época)	17 (14.ª)	15 (12.ª)	2
Tempo de treino	3,8 min	4,1 min	−16 s
Tempo por época (média / mediana)	13,4 s / 11,3 s	16,3 s / 12,9 s	−2,9 s / −1,6 s
Inferência (ms/imagem)	0,156	0,093	0,063
Parâmetros	81 674	61 962	19 712

A diferença entre a LSTM e a GRU (−0,16 p.p.) está dentro da margem de erro: as duas variantes empatam em acurácia, e a GRU consegue-o com 24 % menos parâmetros e uma inferência 40 % mais rápida — 0,093 contra 0,156 ms por imagem (Tabela 11). Nesta execução, porém, a GRU não foi mais rápida a treinar: a mediana foi de 13 s por época, contra 11 s na LSTM, e algumas épocas de ambas foram bastante mais lentas do que as restantes (até 38 s na GRU e 33 s na LSTM), o que aponta para variações de carga da máquina. Em CPU, a vantagem teórica da GRU — três conjuntos de pesos em vez de quatro — não se refletiu no tempo de treino.

Face às outras arquiteturas, como pede o enunciado, a melhor rede recorrente (GRU, 88,61 %) fica 4,01 p.p. abaixo da CNN e 3,53 p.p. abaixo da ResNet50 — diferenças claramente acima da margem de erro —, e praticamente ao nível da MLP da Tarefa 2 (89,16 %), que também não explora a estrutura espacial da imagem. As matrizes de confusão (Figura 10) mostram as mesmas confusões entre peças de parte de cima, mais acentuadas: a camisa continua a ser a pior classe, com um recall de 67,4 % na LSTM e 70,8 % na GRU (71,4 % na CNN).

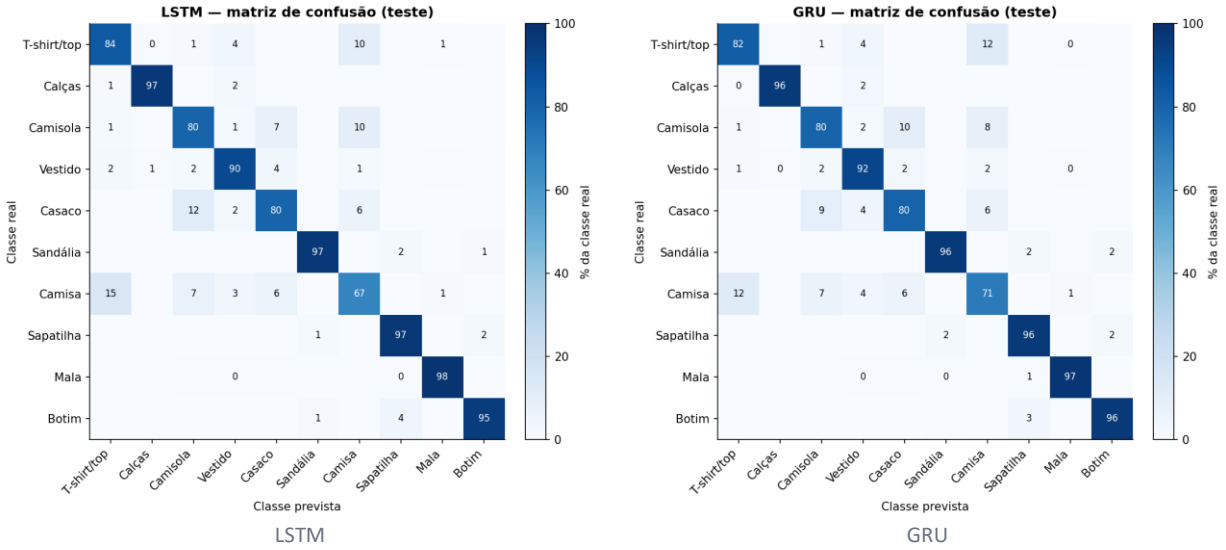


Figura 10 *— Matrizes de confusão das redes recorrentes no teste.*

6.5 Quanto da imagem precisa a RNN de ler?

Como as redes aceitam sequências de qualquer comprimento, é possível pedir-lhes uma classificação a meio da leitura, dando-lhes só as primeiras *k* linhas de cada imagem de teste. As redes nunca foram treinadas para responder a meio da imagem, por isso a curva mostra quanta informação útil o estado interno já acumulou em cada ponto (Figura 11).

Até à 8.ª linha, as duas redes acertam apenas cerca de 20 % — o dobro do acaso, pouco mais do que distinguir as peças que já começaram a aparecer das que ainda não: no calçado, as primeiras linhas estão praticamente vazias, e nas peças de cima a gola e os ombros são muito parecidos entre classes. A partir da 10.ª linha, a acurácia sobe de forma quase linear: com metade da imagem (14 linhas), a LSTM acerta 41,8 % e a GRU 46,0 %; as duas chegam a 90 % da acurácia final ao fim de 24 linhas. As duas curvas são quase indistinguíveis, o que confirma que as duas células guardam a informação de forma equivalente neste problema.

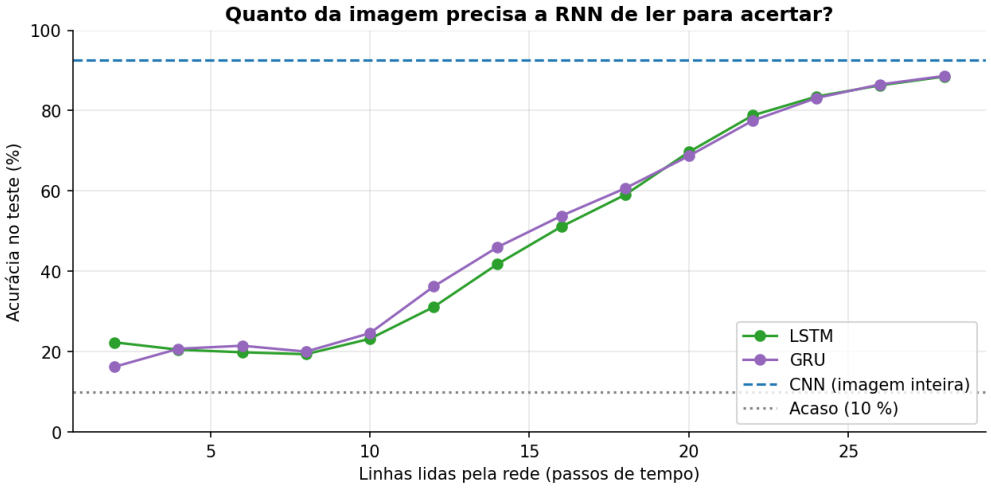


Figura 11 — Acurácia no teste em função do número de linhas lidas; a tracejado, a CNN com a imagem inteira.

É a diferença essencial em relação à CNN, que vê a imagem inteira de uma só vez: a rede recorrente tem de guardar no estado interno tudo o que leu e só consegue decidir com confiança perto do fim. Além disso, partilha pesos entre linhas, mas não ao longo de cada linha — um padrão deslocado alguns píxeis para o lado chega à rede por pesos diferentes, uma invariância que a convolução dá de graça.

7. Parte 4 — Comparação e análise

7.1 Tabela comparativa

A Tabela 12 junta as métricas pedidas no enunciado — acurácia, tempo de treino e tamanho do modelo — com o intervalo de confiança, a perda, o F1 médio e o tempo de inferência. Os quatro modelos foram treinados na mesma máquina, com as mesmas partições e a mesma paragem antecipada, e os tempos só são comparáveis entre si dentro desta execução.

Tabela 12 — Comparação das quatro redes (a negrito, o melhor valor de cada linha). Hardware: CPU (8 núcleos lógicos)

Métrica	CNN	ResNet50	LSTM	GRU
Acurácia no teste	**92,62 %**	92,14 %	88,45 %	88,61 %
IC 95 % (±)	0,51 p.p.	0,53 p.p.	0,63 p.p.	0,62 p.p.
Perda no teste	**0,2180**	0,2241	0,3240	0,3151
F1 médio (macro)	**0,925**	0,921	0,884	0,886
Épocas de treino	19	7	17	15
Tempo de treino	4,8 min	75,6 min	**3,8 min**	4,1 min
Tempo médio por época	15 s	10,8 min	**13 s**	16 s
Parâmetros	421 642	23 608 202	81 674	**61 962**
Tamanho dos pesos	1,61 MB	90,06 MB	0,31 MB	**0,24 MB**
Inferência (ms/imagem)	0,099	1,613	0,156	**0,093**

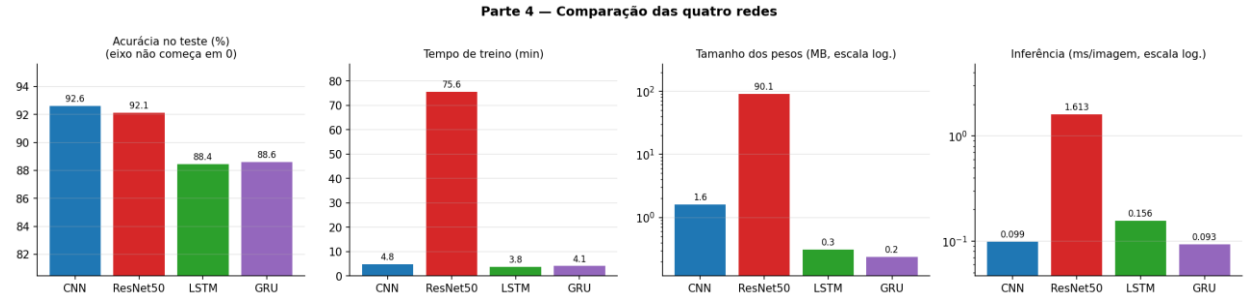


Figura 12 — Acurácia, tempo de treino, tamanho e tempo de inferência das quatro redes (tamanho e inferência em escala logarítmica).

Por ordem de acurácia: CNN (92,62 %), ResNet50 (92,14 %), GRU (88,61 %) e LSTM (88,45 %). Tendo em conta a margem de erro, formam-se dois grupos: a CNN e a ResNet50 empatadas no topo (−0,48 p.p., dentro da margem de ± 0,74 p.p.) e as duas redes recorrentes empatadas cerca de 4 p.p. abaixo.

Nos custos (Figura 12), a ResNet50 destaca-se pela negativa: 75,6 min de treino, contra 3,8–4,8 min das outras três redes; 90 MB de pesos; e 1,61 ms por imagem na inferência, 16× o tempo da CNN. As redes recorrentes são as mais pequenas (0,24–0,31 MB), e a GRU e a CNN as mais rápidas a inferir (cerca de 0,1 ms por imagem).

7.2 Acurácia, dimensão e custo

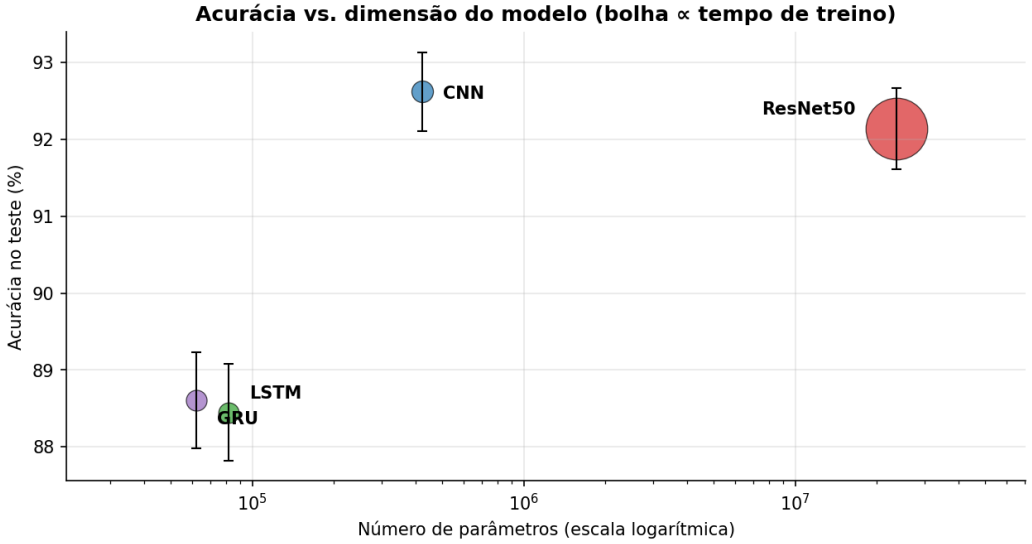


Figura 13 — Acurácia no teste face ao número de parâmetros; a área de cada bolha é proporcional ao tempo de treino e as barras mostram o IC 95 %.

A Figura 13 põe a acurácia em função do número de parâmetros (em escala logarítmica), com a área de cada bolha proporcional ao tempo de treino. A CNN ocupa a posição mais eficiente: está no topo da acurácia com 56× menos parâmetros do que a ResNet50. As redes recorrentes são 5–7 vezes mais pequenas do que a CNN, mas a economia não compensa a perda de acurácia, porque não exploram a estrutura espacial da imagem.

Curiosamente, a CNN, apesar de ter mais parâmetros, treina em tempo semelhante ao das redes recorrentes (15 s por época, contra 13 s e 16 s): as convoluções calculam todas as posições da imagem em paralelo, enquanto as redes recorrentes têm de processar os 28 passos um de cada vez, porque cada estado depende do anterior.

7.3 Desempenho por classe

A Tabela 13 e a Figura 14 mostram o desempenho de cada rede em cada classe.

Tabela 13 *— F1 por classe e por modelo no teste (a negrito, o melhor de cada classe)*

Classe	CNN	ResNet50	LSTM	GRU
T-shirt/top	**0,873**	0,871	0,825	0,832
Calças	0,989	**0,991**	0,975	0,975
Camisola	**0,890**	0,874	0,794	0,805
Vestido	**0,932**	0,930	0,887	0,881
Casaco	**0,886**	0,873	0,806	0,809
Sandália	**0,988**	0,986	0,970	0,965
Camisa	0,767	**0,770**	0,693	0,708
Sapatilha	**0,970**	0,964	0,953	0,951
Mala	0,987	**0,989**	0,977	0,975
Botim	**0,972**	0,967	0,962	0,958
Média (macro)	**0,925**	**0,921**	**0,884**	**0,886**

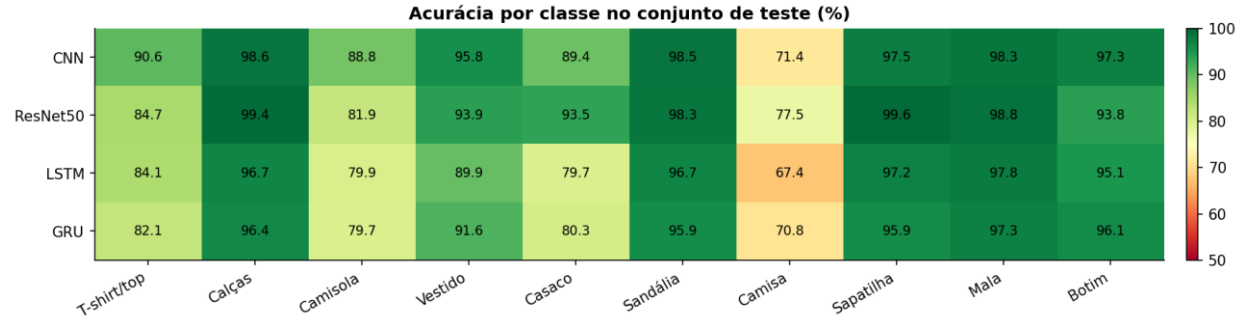


Figura 14 *— Acurácia (recall) por classe e por modelo no conjunto de teste.*

A dificuldade das classes é praticamente a mesma para as quatro redes: **Camisa** é a pior (F1 médio de 0,734), seguida de camisola, casaco e T-shirt/top — peças de parte de cima, com silhuetas muito parecidas. Calças, sandália, sapatilha, mala e botim ficam acima de 0,95 de F1 em todas as redes. As arquiteturas distinguem-se sobretudo nas classes em que a forma da peça não chega e é preciso reparar em pormenores: a maior vantagem da CNN e da ResNet50 sobre as redes recorrentes está em camisola, casaco e camisa.

7.4 Imagens que nenhuma rede acerta

Há 349 imagens de teste (3,5 %) que nenhuma das quatro redes acerta (Figura 15). Entre elas, os pares mais frequentes (classe real → previsão da CNN) são camisa → T-shirt/top (56), camisa → casaco (36) e T-shirt/top → camisa (30). Muitas destas imagens são ambíguas mesmo para uma pessoa — uma camisa sem botões visíveis é, a 28×28 píxeis, uma T-shirt —, e algumas parecem ter a etiqueta errada. Representam, grosso modo, o limite do que se pode esperar deste dataset sem mais informação.



Figura 15 *— Exemplos de imagens de teste que as quatro redes classificam mal, com a classe real e a previsão da CNN.*

7.5 Em que contextos cada arquitetura é mais vantajosa

Os resultados deste trabalho são um caso particular. A Tabela 14 generaliza-os: resume, para cada arquitetura, os cenários em que tende a ser a melhor escolha e aqueles em que não é.

Tabela 14 *— Contextos em que cada arquitetura é mais (e menos) vantajosa*

Arquitetura	Mais vantajosa quando…	Menos indicada quando…
CNN simples	• Imagens pequenas ou de baixa resolução, com enquadramento pouco variável: documentos digitalizados, ícones, peças numa linha de produção com câmara fixa, imagens de sensores. • Dispositivos com pouca memória ou sem GPU (telemóveis, microcontroladores, câmaras inteligentes): poucos parâmetros, 1–2 MB de pesos e inferência rápida em CPU. • Domínios diferentes das fotografias naturais (espectrogramas, imagens médicas em tons de cinzento, radar), quando há dados rotulados suficientes para treinar de raiz. • Como primeira *baseline*: treina em minutos e mostra se vale a pena investir numa rede maior.	• Poucos dados rotulados: sem pesos pré-treinados, sobreajusta. • Imagens grandes e variadas (fundos, escalas, poses), que pedem mais profundidade e campo recetivo.
ResNet pré-treinada (*transfer learning*)	• Poucas imagens rotuladas (de centenas a alguns milhares): as características do ImageNet compensam a falta de dados. • Fotografias a cores de resolução média ou alta (≥ 96–224 píxeis), com objetos, texturas e fundos variados. • Tarefas difíceis: muitas classes, distinções finas (espécies, defeitos subtis) ou como *backbone* de deteção e segmentação (Faster R-CNN, Mask R-CNN, U-Net com codificador ResNet).	• Sem GPU, ou com limites apertados de memória e latência: ~90 MB de pesos e inferência muito mais lenta. • Imagens pequenas e muito diferentes do ImageNet, como neste trabalho: a transferência é parcial e o custo não se paga.
RNN (LSTM / GRU)	• Dados sequenciais por natureza: texto, fala e áudio, séries temporais (sensores, consumos, preços), sinais biomédicos (ECG), registos de eventos. • Em visão, quando a imagem esconde uma sequência: texto manuscrito ou matrículas (CRNN — uma CNN extrai características e uma LSTM lê-as da esquerda para a direita), vídeo (CNN por frame + LSTM no tempo) ou legendagem de imagens. • GRU quando os recursos contam (menos parâmetros, treino mais rápido); LSTM quando as dependências são muito longas.	• Classificação de imagens estáticas: vê uma linha de cada vez, não tem partilha de pesos ao longo de cada linha e o treino é sequencial, logo difícil de paralelizar. • Sequências muito longas, em que os *Transformers* já as substituíram.

Neste trabalho, o Fashion-MNIST favorece claramente a CNN: imagens pequenas, centradas, em tons de cinzento e com muitos dados rotulados. A ResNet50 esteve fora do seu terreno — imagens de 28×28 ampliadas para 32×32, muito longe das fotografias do ImageNet — e, ainda assim, empatou com a CNN graças à afinação; as redes recorrentes provaram que é possível classificar imagens como sequências, mas não que seja boa ideia.

8. Conclusões

1. **As quatro redes cumprem o enunciado e classificam bem o Fashion-MNIST**: CNN com 92,62 %, ResNet50 com 92,14 %, LSTM com 88,45 % e GRU com 88,61 % de acurácia no teste. A CNN e a ResNet50 ficam cerca de 3 p.p. acima da MLP da Tarefa 2 (89,16 %); as redes recorrentes ficam ao nível dela.
2. **A estrutura da rede pesa mais do que o tamanho.** A CNN aproveita a localidade dos padrões e a partilha de pesos entre posições e, com 421 642 parâmetros e 4,8 min de treino, foi a mais precisa e a que oferece o melhor equilíbrio entre acurácia e custo.

3. **O *transfer learning* depende da proximidade entre domínios.** As características da ResNet50 transferem-se mal para silhuetas cinzentas de 32×32 (83,77 % de validação com a base congelada), e é a afinação da base inteira que as adapta. Mesmo assim, o resultado final empata com o da CNN (−0,48 p.p., dentro da margem de erro), com 56× os parâmetros e 15,7× o tempo de treino.
4. **Uma RNN consegue classificar imagens, mas não é a ferramenta natural para isso.** A melhor rede recorrente (GRU) fica 4,0 p.p. abaixo da CNN, e a sua acurácia só se aproxima do valor final nas últimas linhas lidas. A LSTM e a GRU empatam, e a GRU consegue-o com 24 % menos parâmetros.
5. **A acurácia sozinha não chega para escolher um modelo.** O tamanho, o tempo de treino e a latência de inferência decidem o que é viável em produção; e diferenças de acurácia dentro da margem de erro não justificam trocar uma rede pequena por uma dezenas de vezes maior.

Limitações e próximos passos

- Uma única execução por modelo: repetir com várias sementes daria intervalos de confiança mais sólidos para as diferenças pequenas (o IC usado trata os modelos como independentes, apesar de avaliados nas mesmas imagens; um teste de McNemar seria mais rigoroso).
- Sem aumento de dados (*data augmentation*): pequenas rotações e deslocações costumam melhorar a CNN e a ResNet50.
- A ResNet50 foi usada a 32×32 por limitações de CPU (a afinação levou 71,8 min); com GPU, uma resolução de 64–96 píxeis deverá aproximar mais as características das do ImageNet. Mais épocas, sozinhas, não chegariam: a validação já piorou na última época da afinação.
- A CNN e a ResNet50 erram de formas um pouco diferentes (a ResNet50 troca mais camisolas por casacos); combinar as duas previsões (*ensemble*) poderia ganhar alguns décimos de ponto.
- Uma arquitetura híbrida CNN + LSTM, ou um *Vision Transformer*, seria o passo natural para explorar a leitura sequencial sem perder a informação espacial.

Anexo A — Capturas complementares do notebook

Estas capturas mostram algumas saídas tal como aparecem no notebook. Os mesmos valores estão organizados nas tabelas do relatório.

In [8]:
```
# 2.2 Treino
hist_cnn, tempo_cnn, tempos_cnn = treinar(cnn, X_TREINO, y_treino, X_VAL, y_val,
                                          epocas=EPOCAS_MAX, lote=128)
print(f"\nTreino da CNN: {tempo_cnn:.0f} s em {len(tempos_cnn)} épocas "
      f"({np.mean(tempos_cnn):.1f} s por época)")
```

```
Epoch 1/20
422/422 - 15s - 35ms/step - accuracy: 0.8025 - loss: 0.5505 - val_accuracy: 0.8805 - val_loss: 0.3337
Epoch 2/20
422/422 - 15s - 35ms/step - accuracy: 0.8730 - loss: 0.3536 - val_accuracy: 0.8915 - val_loss: 0.2916
Epoch 3/20
422/422 - 15s - 36ms/step - accuracy: 0.8859 - loss: 0.3097 - val_accuracy: 0.9113 - val_loss: 0.2463
Epoch 4/20
422/422 - 14s - 34ms/step - accuracy: 0.8981 - loss: 0.2796 - val_accuracy: 0.9155 - val_loss: 0.2350
Epoch 5/20
422/422 - 14s - 34ms/step - accuracy: 0.9049 - loss: 0.2570 - val_accuracy: 0.9167 - val_loss: 0.2253
Epoch 6/20
422/422 - 15s - 36ms/step - accuracy: 0.9106 - loss: 0.2407 - val_accuracy: 0.9215 - val_loss: 0.2192
Epoch 7/20
422/422 - 15s - 35ms/step - accuracy: 0.9158 - loss: 0.2264 - val_accuracy: 0.9258 - val_loss: 0.2028
Epoch 8/20
422/422 - 15s - 35ms/step - accuracy: 0.9213 - loss: 0.2133 - val_accuracy: 0.9262 - val_loss: 0.2017
Epoch 9/20
422/422 - 14s - 34ms/step - accuracy: 0.9261 - loss: 0.2006 - val_accuracy: 0.9295 - val_loss: 0.1957
Epoch 10/20
422/422 - 16s - 38ms/step - accuracy: 0.9291 - loss: 0.1901 - val_accuracy: 0.9268 - val_loss: 0.2012
Epoch 11/20
422/422 - 15s - 36ms/step - accuracy: 0.9317 - loss: 0.1818 - val_accuracy: 0.9297 - val_loss: 0.1906
Epoch 12/20
422/422 - 19s - 44ms/step - accuracy: 0.9362 - loss: 0.1698 - val_accuracy: 0.9275 - val_loss: 0.1945
Epoch 13/20
422/422 - 15s - 36ms/step - accuracy: 0.9386 - loss: 0.1663 - val_accuracy: 0.9295 - val_loss: 0.1910
Epoch 14/20
422/422 - 15s - 35ms/step - accuracy: 0.9398 - loss: 0.1589 - val_accuracy: 0.9302 - val_loss: 0.1893
Epoch 15/20
422/422 - 15s - 35ms/step - accuracy: 0.9432 - loss: 0.1535 - val_accuracy: 0.9327 - val_loss: 0.1898
Epoch 16/20
422/422 - 15s - 36ms/step - accuracy: 0.9460 - loss: 0.1449 - val_accuracy: 0.9313 - val_loss: 0.1874
Epoch 17/20
422/422 - 15s - 36ms/step - accuracy: 0.9464 - loss: 0.1386 - val_accuracy: 0.9318 - val_loss: 0.1876
Epoch 18/20
422/422 - 16s - 37ms/step - accuracy: 0.9493 - loss: 0.1356 - val_accuracy: 0.9325 - val_loss: 0.1885
Epoch 19/20
422/422 - 15s - 36ms/step - accuracy: 0.9510 - loss: 0.1277 - val_accuracy: 0.9303 - val_loss: 0.1903
Treino da CNN: 289 s em 19 épocas (15.2 s por época)
```

Captura 5 *— registo do treino da CNN: uma linha por época, até à paragem antecipada.*

In [10]:
```
# 2.4 Avaliação no conjunto de teste
prev_cnn = avaliar("CNN", cnn, X_TESTE, y_teste, hist_cnn, tempo_cnn, tempos_cnn)
print()
print(classification_report(y_teste, prev_cnn, target_names=CLASSES, digits=3, zero_division=0))
grafico_confusao(y_teste, prev_cnn, "CNN — matriz de confusão (teste)", "03_cnn_confusao.png")
```

```
CNN: acurácia no teste 92.62% (± 0.51%) | perda 0.2180 | 421,642 parâmetros | 1.61 MB | 0.099 ms por imagem

              precision    recall  f1-score   support

 T-shirt/top      0.843     0.906     0.873      1000
      Calças      0.992     0.986     0.989      1000
    Camisola      0.892     0.888     0.890      1000
     Vestido      0.907     0.958     0.932      1000
      Casaco      0.877     0.894     0.886      1000
    Sandália      0.991     0.985     0.988      1000
      Camisa      0.827     0.714     0.767      1000
   Sapatilha      0.965     0.975     0.970      1000
        Mala      0.991     0.983     0.987      1000
       Botim      0.971     0.973     0.972      1000

    accuracy                          0.926     10000
   macro avg      0.926     0.926     0.925     10000
weighted avg      0.926     0.926     0.925     10000
```

Captura 6 *— avaliação da CNN no conjunto de teste (`classification_report`).*

In [15]:
```
# 3.4 Fase B — afinação: descongelar a base e treinar com uma taxa de aprendizagem 10x menor
EPOCAS_FASE_B = 1 if MODO_RAPIDO else 5
INICIO_AFINACAO = None   # None = toda a base; "conv5_block1" = só o último estágio (≈ metade do tempo em CPU)

base_resnet.trainable = True
descongelar = INICIO_AFINACAO is None
for camada in base_resnet.layers:
    if INICIO_AFINACAO and camada.name.startswith(INICIO_AFINACAO):
        descongelar = True
    # As BatchNormalization ficam congeladas: mantêm as médias e variâncias aprendidas no ImageNet
    camada.trainable = descongelar and not isinstance(camada, layers.BatchNormalization)

# Mudar `trainable` só tem efeito depois de voltar a compilar
resnet.compile(optimizer=keras.optimizers.Adam(1e-4), loss="sparse_categorical_crossentropy",
               metrics=["accuracy"])
print(f"Parâmetros treinados na fase B: {sum(np.prod(p.shape) for p in resnet.trainable_weights):,}")

hist_b, tempo_b, tempos_b = treinar(resnet, X_TREINO, y_treino, X_VAL, y_val,
                                    epocas=EPOCAS_FASE_B, lote=64, paciencia=2)

# Juntar as duas fases num único histórico
hist_resnet = {chave: hist_a[chave] + hist_b[chave] for chave in hist_a}
tempo_resnet, tempos_resnet = tempo_a + tempo_b, tempos_a + tempos_b
print(f"\nResNet50: {tempo_resnet:.0f} s no total (fase A: {tempo_a:.0f} s; fase B: {tempo_b:.0f} s)")
```

```
Parâmetros treinados na fase B: 23,501,962
Epoch 1/5
844/844 - 936s - 1s/step - accuracy: 0.8747 - loss: 0.3521 - val_accuracy: 0.9028 - val_loss: 0.2680
Epoch 2/5
844/844 - 891s - 1s/step - accuracy: 0.9163 - loss: 0.2345 - val_accuracy: 0.9187 - val_loss: 0.2262
Epoch 3/5
844/844 - 761s - 902ms/step - accuracy: 0.9290 - loss: 0.1945 - val_accuracy: 0.9265 - val_loss: 0.2137
Epoch 4/5
844/844 - 768s - 910ms/step - accuracy: 0.9400 - loss: 0.1633 - val_accuracy: 0.9298 - val_loss: 0.1996
Epoch 5/5
844/844 - 949s - 1s/step - accuracy: 0.9486 - loss: 0.1401 - val_accuracy: 0.9273 - val_loss: 0.2187
ResNet50: 4538 s no total (fase A: 232 s; fase B: 4306 s)
```

Captura 7 *— fase B da ResNet50: descongelamento da base (sem as camadas BatchNormalization) e registo da afinação.*

In [24]:
```
# ============================================================================
# 5.1  Tabela comparativa
# ============================================================================
modelos = list(RESULTADOS)
tabela = pd.DataFrame(RESULTADOS).T.astype(float)
tabela.index.name = "Modelo"
tabela["Treino (× CNN)"] = tabela["Tempo de treino (s)"] / tabela.loc["CNN", "Tempo de treino (s)"]
tabela["Tamanho (× CNN)"] = tabela["Tamanho dos pesos (MB)"] / tabela.loc["CNN", "Tamanho dos pesos (MB)"]
tabela.to_csv(os.path.join(PASTA, "tabela_comparativa.csv"))

formatos = {
    "Acurácia (teste)": "{:.2%}", "IC 95 % (±)": "{:.2%}", "Perda (teste)": "{:.4f}",
    "Épocas": "{:.0f}", "Tempo de treino (s)": "{:,.0f}", "Tempo médio por época (s)": "{:,.1f}",
    "Parâmetros": "{:,.0f}", "Parâmetros treináveis": "{:,.0f}", "Tamanho dos pesos (MB)": "{:.2f}",
    "Inferência (ms/imagem)": "{:.3f}", "Treino (× CNN)": "{:.1f}×", "Tamanho (× CNN)": "{:.1f}×",
}
print("Hardware:", DISPOSITIVO)
display(pd.DataFrame({coluna: tabela[coluna].map(formato.format) for coluna, formato in formatos.items()}).T)
```

```
Hardware: CPU (8 núcleos lógicos)
```

Modelo	CNN	ResNet50	LSTM	GRU
Acurácia (teste)	92.62%	92.14%	88.45%	88.61%
IC 95 % (±)	0.51%	0.53%	0.63%	0.62%
Perda (teste)	0.2180	0.2241	0.3240	0.3151
Épocas	19	7	17	15
Tempo de treino (s)	289	4,538	229	245
Tempo médio por época (s)	15.2	647.9	13.4	16.3
Parâmetros	421,642	23,608,202	81,674	61,962
Parâmetros treináveis	421,642	23,501,962	81,674	61,962
Tamanho dos pesos (MB)	1.61	90.06	0.31	0.24
Inferência (ms/imagem)	0.099	1.613	0.156	0.093
Treino (× CNN)	1.0×	15.7×	0.8×	0.8×
Tamanho (× CNN)	1.0×	56.0×	0.2×	0.1×

Captura 8 *— tabela comparativa final, tal como é apresentada no notebook.*

In [29]:
```
# 5.7 Resumo automático dos resultados desta execução
acc = tabela["Acurácia (teste)"]
ic = tabela["IC 95 % (±)"]
mais_precisa = acc.idxmax()
rnn_melhor = acc[["LSTM", "GRU"]].idxmax()


def diferenca_texto(a, b):
    """Diferença de acurácia entre dois modelos, indicando se é estatisticamente clara."""
    d = (acc[a] - acc[b]) * 100
    margem = np.sqrt(ic[a] ** 2 + ic[b] ** 2) * 100   # margem aproximada da diferença (IC 95 %)
    return f"{d:+.2f} p.p. ({'diferença clara' if abs(d) > margem else 'dentro da margem de erro'})"


resumo = [
    f"Modelo mais preciso: {mais_precisa} ({acc[mais_precisa]:.2%} ± {ic[mais_precisa]:.2%}).",
    f"ResNet50 vs. CNN: {diferenca_texto('ResNet50', 'CNN')}, com "
    f"{tabela.loc['ResNet50', 'Parâmetros'] / tabela.loc['CNN', 'Parâmetros']:.0f}× mais parâmetros, "
    f"{tabela.loc['ResNet50', 'Tamanho (× CNN)']:.0f}× o tamanho e "
    f"{tabela.loc['ResNet50', 'Treino (× CNN)']:.1f}× o tempo de treino da CNN.",
    f"Melhor RNN ({rnn_melhor}) vs. CNN: {diferenca_texto(rnn_melhor, 'CNN')}.",
    f"LSTM vs. GRU: {diferenca_texto('LSTM', 'GRU')}; a GRU tem "
    f"{1 - tabela.loc['GRU', 'Parâmetros'] / tabela.loc['LSTM', 'Parâmetros']:.0%} menos parâmetros.",
    f"Tempo total do notebook: {(time.perf_counter() - INICIO_NOTEBOOK) / 60:.1f} min em {DISPOSITIVO}.",
]
print("\n".join("• " + linha for linha in resumo))
```

```
• Modelo mais preciso: CNN (92.62% ± 0.51%).
• ResNet50 vs. CNN: -0.48 p.p. (dentro da margem de erro), com 56× mais parâmetros, 56× o tamanho e 15.7× o tempo de treino da CNN.
• Melhor RNN (GRU) vs. CNN: -4.01 p.p. (diferença clara).
• LSTM vs. GRU: -0.16 p.p. (dentro da margem de erro); a GRU tem 24% menos parâmetros.
• Tempo total do notebook: 89.2 min em CPU (8 núcleos lógicos).
```

Captura 9 *— resumo automático dos resultados, gerado no fim do notebook.*